

Documento Processo de Teste

Projeto: TOT — Pinhalense Auxílio de Pesquisa e Automatização

Tecnologia: Flutter (Dart) + Firebase + Groq API (Llama 4 Scout)

Arquitetura: Stateful Widgets + Services + Backend Firebase

Norma aplicada: ISO/IEC/IEEE 29119-2

1. Estratégia de Teste

- Testes de unidade (sanitizer, IaService, ViewModels lógicos das telas)
- Testes de integração (fluxos completos Login → Home, Croqui → IA)
- Uso de FakeIaService (substitui chamada HTTP por respostas controladas)
- Uso de FakeFirebaseStorage e FakeFirestore (pacotes firebase_auth_mock e fake_cloud_firestore)
- Mock HTTP via package:http/testing.dart (MockClient)

2. Ambiente de Teste

- Flutter SDK ($\geq 3.0.0$)
- Dart SDK ($\geq 3.0.0$)
- flutter_test
- integration_test
- mocktail (mocks dos serviços)
- firebase_auth_mock
- fake_cloud_firestore
- http (MockClient)

3. Critérios de Entrada

- Projeto compila sem erros (flutter analyze)
- Telas de Login, Cadastro, Home e AdminBanco implementadas
- IaService implementado e isolado em lib/Utils/
- sanitize() implementado em lib/Utils/sanitizer.dart
- Documento A concluído e aprovado

4. Critérios de Saída

- Todos os casos de teste de CT01 a CT17 executados
- Resultados de aprovado/reprovado registrados
- Cobertura mínima de 70% nos arquivos sanitizer.dart e ia_service.dart
- Relatório de execução produzido
- Nenhum risco crítico (R01, R04) em aberto sem mitigação

5. Ordem de Execução

1. Testar sanitize() (unidade)
2. Testar IaService (unidade, com MockClient)
3. Testar lógica de validação do LoginScreen
4. Testar lógica de validação do RegisterPage
5. Testar AdminBancoPage (adicionar/remover com fakes)

6. Testar integração Cadastro → Login
7. Testar integração Login → Home
8. Testar integração Envio de croqui → resposta da IA

6. Implementação

```
test/  
  unit/  
    sanitizer_test.dart  
    ia_service_test.dart  
    login_validation_test.dart  
    register_validation_test.dart  
    admin_banco_test.dart  
  integration_test/  
    auth_flow_test.dart          // Cadastro → Login → Home  
    ia_flow_test.dart           // Croqui → IaService → exibição da imagem  
    banco_imagens_flow_test.dart // Admin: adicionar e remover
```

7. Controle

- Planejados: 17
- Executados
- Aprovados
- Reprovados
- Bloqueados

8. Execução

```
flutter test test/unit  
flutter test integration_test  
flutter test --coverage
```

9. Conclusão

- Encerrar após execução completa, análise dos resultados e fechamento dos riscos R01 e R04 com plano de mitigação documentado.